Universidad Peruana Cayetano Heredia
Facultad de Ciencias e Ingeniería
Departamento de Ciencias Exactas

Programación Avanzada 2025

Lab. 8.2. Análisis de Algoritmos. Análisis Asintótico

Octubre 11, 2025

Cree una carpeta (folder) en el disco D, nómbrela con su apellido paterno seguido de su código. Ejemplo: LOPEZ12345

Estructura de Datos

Es una manera sistemática de organizar, gestionar, y almacenar datos, a fin de usarlos de manera eficiente.

Ejemplos de Estructuras de Datos:
- Array
- Listas enlazadas
- Pilas
- Colas
- Árboles
- Grafos

Algoritmo

Procedimiento paso a paso para ejecutar alguna tarea en una cantidad finita de tiempo.

Los algoritmos describen procesos que se ejecutan en computadoras reales que tienen recursos finitos. Los procesos consumen principalmente dos recursos: tiempo y espacio (tiempo de ejecución y memoria).

Evaluar la **eficiencia de los algoritmos** es importante en la industria del software, en especial los algoritmos que implementan las estructuras de datos básicas.

Desde el punto de vista de la estructura de datos, las siguientes categorías de algoritmos son las más importantes:
- Búsqueda
- Ordenación
- Insertar
- Actualizar
- Eliminar

Midiendo la Eficiencia de los Algoritmos. Medición del Tiempo de Ejecución.

Una de las técnicas para medir el costo de tiempo de los algoritmos es usar el reloj del computador para obtener el tiempo de ejecución, esta técnica se llama **benchmarking**. Se inicia midiendo el tiempo para un conjunto de datos, luego se mide para conjuntos de datos más grandes. Después de varias pruebas, podemos lograr datos suficientes para predecir el comportamiento del algoritmo para cualquier conjunto de datos de cualquier tamaño.

Ejemplo:

Elabore un programa que cuente desde 1 hasta un número dado. Este número será también el tamaño del problema. Luego, duplique el tamaño del problema y repita el proceso. Después de cinco incrementos, tendremos un conjunto de resultados del cual podemos generalizar. He aquí el programa de prueba que inicia con un tamaño de problema igual a 10000000.

```
import time
problemSize = 10000000
print(f"{'Problem Size':12}{'Seconds':>16}")
for count in range(5):
      start = time.time()
      # Inicio del algoritmo
      work = 1
      for x in range(problemSize):
         work += 1
         work -= 1
      # Fin del algoritmo
      elapsed = time.time() - start
      print(f"{problemSize:12}{elapsed:16.3f}")
      problemSize *= 2     # tamaño del problema se duplica en cada vuelta
```

1. ¿qué captura la función `time()`?
2. Ejecute el programa y analice los resultados, ¿cómo varía el tiempo de ejecución a medida que el tamaño del problema se duplica?, ¿se podría predecir el tiempo de ejecución para un problema de tamaño 32,000,000?
3. Ejecute varias veces el programa, ¿por qué en cada ejecución muestra distintos resultados para un mismo tamaño de problema?
4. Verifique cómo varía el tiempo de ejecución si eliminan uno de los trabajos (esfuerzos) del algoritmo (una de las instrucciones que está dentro del bucle interno)
5. Modifique el programa colocando el esfuerzo del algoritmo dentro de dos bucles anidados, de la siguiente manera:

```
work = 1
for j in range(problemSize):
      for k in range(problemSize):
            work += 1
            work -= 1
```

6. Ejecute el programa, espere 10 segundos, si no muestra resultados, cancele la ejecución. ¿a qué se debe ése comportamiento?
7. En el programa del ejercicio N° 5, modifique el tamaño inicial del problema a 1000, luego ejecute el programa y analice los resultados, ¿cómo varía el tiempo de ejecución a medida que el tamaño del problema se duplica?
8. Analizando los resultados del ejercicio anterior, ¿cuánto tiempo tomará procesar un problema de tamaño 16,000,000?

La medición del tiempo de ejecución a fin de evaluar la eficiencia de los algoritmos, es buena para muchos algoritmos, sin embargo, tiene dos problemas importantes:
- El tiempo de ejecución, para un mismo algoritmo, es dependiente de la plataforma, tanto del hardware como del software.
- Para ciertos algoritmos con tamaño de problema bastante grande, no es práctico medir el tiempo de ejecución.

Por lo expuesto, sería bueno encontrar otras técnicas que no dependan ni del hardware ni del software.

Midiendo la Eficiencia de los Algoritmos. Contando Instrucciones.

Otra técnica de medir la eficiencia de los algoritmos es contar la cantidad de instrucciones ejecutadas para diferentes tamaños de los problemas.

Está técnica es independiente de la plataforma. Tener en cuenta que la contabilidad es sobre las instrucciones de alto nivel no sobre las de bajo nivel.

En esta técnica podemos distinguir dos tipos de instrucciones:
- Las que se ejecutan la misma cantidad de veces sin importar el tamaño del problema.
- Las que su contabilidad de ejecución varía con el tamaño del problema.

Podemos ignorar las del primer tipo debido a que no influyen significativamente en este tipo de análisis.
Las del segundo tipo las encontramos principalmente en los bucles y en las funciones recursivas. Para el caso de bucles, bastaría contar la cantidad de iteraciones.

9. Modifique el programa del ejercicio N° 7 a fin de que muestre, en lugar del tiempo, la cantidad de iteraciones del algoritmo para cada conjunto de datos.
10. Analizando los resultados del ejercicio anterior, ¿cómo varía la cantidad de instrucciones a medida que varía el tamaño del problema?
11. Escriba un programa de prueba que cuente y muestre el número de iteraciones del siguiente bucle, ¿qué pasa con el número de iteraciones a medida que el tamaño del problema se duplica?

```
while problemSize > 0:
      problemSize = problemSize // 2
```

Contar las instrucciones es muy buena idea, sin embargo, sería mejor si tuviéramos una técnica analítica para un completo método de análisis.

Análisis de Complejidad

Está técnica analítica permite evaluar la eficiencia de los algoritmos, sin la necesidad de medir el tiempo de ejecución y sin la necesidad de contar la cantidad de instrucciones. Para esta técnica basta lápiz y papel.

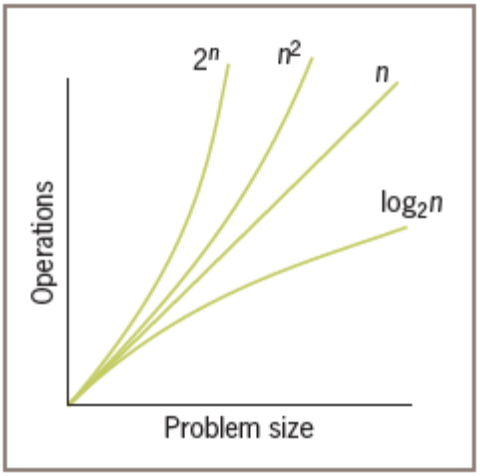
12. Analizando los algoritmos de los ejercicios 2 y 9 rellene la siguiente tabla, la palabra "trabajo" indica número de iteraciones.

Tamaño del Problema	Trabajo del Primer Algoritmo	Trabajo del Segundo Algoritmo
2		
10		
1000		

13. ¿Cuál sería la gráfica número de instrucciones vs tamaño del problema para cada algoritmo?

El desempeño de los algoritmos difiere en lo que se llama **orden de complejidad**. El primer algoritmo tiene un desempeño **lineal**, su trabajo crece en proporción directa al tamaño del problema. El desempeño del segundo algoritmo es **cuadrático**, su trabajo crece como una función del cuadrado del tamaño del problema. A medida que el tamaño del problema crece, el desempeño de los algoritmos de mayor complejidad empeora.

Existen distintos órdenes de complejidad usados en el análisis de algoritmos.



n	Logarítmico ($\log_2 n$)	Lineal (n)	Cuadrático (n^2)	Exponencial (2^n)
100	7	100	10,000	Fuera del cuadro
1,000	10	1,000	1,000,000	Fuera del cuadro
1,000,000	20	1,000,000	1,000,000,000,000	Fuera del cuadro

La siguiente tabla muestra las siete funciones usadas en el análisis de algoritmos.

Constante	logarítmico	lineal	n-log-n	cuadrático	cúbico	exponencial
1	$\log_2 n$	n	$n\log_2 n$	n^2	n^3	a^n

La Notación Big-*O*

La notación Big-O es una notación especial que nos indica cuán fuerte debe trabajar un algoritmo para resolver un problema.

De acuerdo a las funciones usadas en el análisis de algoritmos, tenemos:
$O(1)$, $O(\log n)$, $O(n)$, $O(n\log n)$, $O(n^2)$, $O(n^3)$, $O(a^n)$

La cantidad de operaciones normalmente no coinciden exactamente con n, n^2, o a^n debido a que los algoritmos realizan trabajos extras antes, dentro y después de los bucles. Por ejemplo podemos decir que los algoritmos, en realidad ejecutan la cantidad de:
- $2n + 3$
- $2n^2$
- $\frac{1}{2} n^2 + \frac{1}{2} n$

Si la cantidad de operaciones se expresa en forma de polinomio, basta en centrarnos en el término **dominante**. En general el término dominante es aquel que tiene el mayor exponente. Por ejemplo en la expresión $3n^2+5n-10$, el término dominante es $3n^2$. Este análisis se suele llamar **asintótico**, debido a que el valor del polinomio se acerca al valor del término dominante a medida que el tamaño del problema crece.

14. Asuma que cada una de las siguientes expresiones indica el número de operaciones ejecutadas por un algoritmo para un problema de tamaño **n**. Indique cuál es el término dominante de cada algoritmo, y use la notación big-*O* para clasificarlos.
    a. $2^n - 4n^2 + 5n$
    b. $3n^2 + 6$
    c. $n^3 + n^2 - n$

15. Un algoritmo cuadrático con tiempo de procesamiento $T(n)=cn^2$ demora T(N) segundos para procesar N ítem de datos. ¿Cuánto se demorará para procesar n=5000 ítems de datos, asumiendo que N=100 y T(N)=1ms?

16. Un algoritmo con complejidad de tiempo $O(f(n))$ y tiempo de procesamiento $T(n)=cf(n)$, donde f(n) es una función conocida de n, demora 10 segundos para procesar 1000 ítems de datos. ¿Cuánto tiempo demorará para procesar 100,000 ítems de datos si $f(n)=n$ y $f(n)=n^3$?

17. Determinar la complejidad computacional de los siguientes códigos:

(a)
```
def fun9(self, n):
    m = 0
    i = n
    while i > 0:
        j = 0
        while j< i:
            m += 1
            j += 1
        i /= 2
    return m
```

(b)
```
def fun10(self, n):
    m = 0
    i = 0
    while i < n:
        j = i
        while j> 0:
            m += 1
            j -= 1
        i += 1
    return m
```

(c)
```
def fun12(self, n):
    m = 0
    i = 0
    while i < n:
        j = 0
        while j< n:
            m += 1
            j += 1
        i += 1
    return m
```